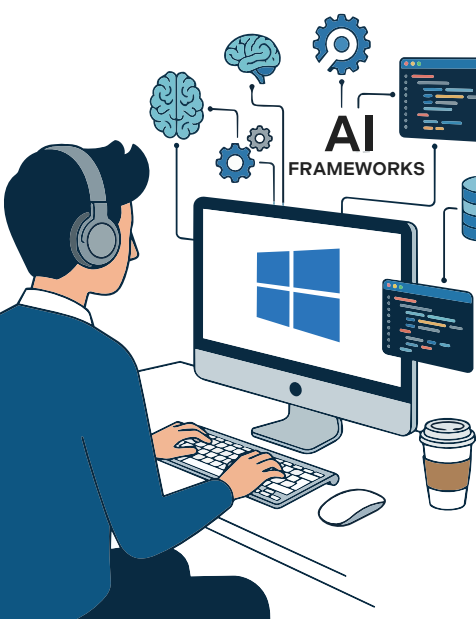


Working with Open Source AI Frameworks on Windows

Open source libraries like PyTorch and TensorFlow have made hardware-independent AI development simpler and are enabling high-performance computing on Windows.



Historically, the Windows operating system has been viewed as a suboptimal choice for high-performance artificial intelligence (AI) workloads, largely due to limited support for GPU computing and research-oriented frameworks. Linux-based systems are the ones mostly chosen for AI development environments by academia and enterprises. However, this perception has changed a lot in the recent past.

Open source AI frameworks have come a long way, the GPU acceleration technologies have improved a lot, and the introduction of the Windows Subsystem for Linux (WSL) have all played a part in making Windows a viable and competitive platform for AI experimentation and deployment. Compatibility with deep learning workflows is now one of the features

that Windows boasts, while it maintains usability, enterprise integration, and developer accessibility. Consequently, AI practitioners can work on the whole cycle of a model from development to testing to deployment on Windows and do not have to rely on an OS or infrastructure.

The backbone of this transition is made up of different open source libraries such as PyTorch, TensorFlow, ONNX Runtime, and scikit-learn. These libraries help to overcome vendor lock-in and make research reproducible. They also make hardware-independent AI development a more straightforward task. A real AI computation pipeline can now be run on Windows efficiently, demonstrating that Windows is not only compatible with open source frameworks but also capable of handling meaningful tensor operations.

Here's an example of an open source AI stack on Windows:

```
import torch
import platform
print("Operating System:", platform.system())
print("PyTorch Version:", torch.__version__)
print("CUDA Available:", torch.cuda.is_available())
```

High-performance AI models on Windows for constrained hardware

A common challenge in AI development is executing computationally extensive models on systems with constrained hardware resources. Most of the Windows-based systems that students,

researchers, and practitioners have access to operate with low GPU memory, small CPU cores, and limited power budgets. There are several methods of working around these limitations to enhance the performance of AI processes:

- Use of pre-trained and transfer-learning models, doing away with the need for starting training.
- Carrying out computations with mixed precision, so that memory consumption is reduced and inference gets faster.
- Removing some parts of the model through pruning and quantization, thus shrinking size and lowering costs of computations.
- Using efficient inference engines like ONNX Runtime and optimised PyTorch backends.

These methods help AI applications developed for high-end computers run even on everyday PCs running Windows, as the memory and computational overhead are significantly reduced while the standards of performance are maintained.

Numerous ways of optimisation allow the execution of demanding AI workloads on limited Windows hardware with good performance. As an example, you can use the following code to deploy a compact CNN for efficient inference:

```
import torch
from torchvision.models import mobilenet_v2
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```